

Quicksort

Quicksort is a divide-and-conquer algorithm that sorts an array by partitioning it around a chosen element, the *pivot*, into two groups: elements smaller than the pivot and elements larger than it. The pivot itself lands in its final sorted position. The two subarrays are then sorted recursively, and because partitioning places the pivot correctly, no merging step is needed afterward — unlike mergesort.

Partitioning. Given array $A[lo..hi]$, pick a pivot value p (commonly $A[hi]$). Scan the array, moving all elements $\leq p$ to the left side and all elements $> p$ to the right, then place p between them. This takes $\Theta(n)$ time for a subarray of size n , using only constant extra space (in-place).

Worked example. Sort $[7, 2, 9, 4, 1]$ using the last element as pivot:

- Pivot = 1. Nothing is smaller, so 1 stays at index 0: $[1, 7, 2, 9, 4]$.
- Recurse on $[7, 2, 9, 4]$, pivot = 4: elements ≤ 4 are $\{2\}$, so partition gives $[2, 4, 7, 9]$... continuing recursively on $[7, 9]$ eventually yields the sorted array $[1, 2, 4, 7, 9]$.

Each partition step fixes one element's final position, and the recursion tree shrinks the problem size until subarrays of length ≤ 1 are trivially sorted.

Complexity analysis. If the pivot always splits the array evenly, the recurrence is

$$T(n) = 2T(n/2) + \Theta(n),$$

which solves by the Master Theorem to $T(n) = \Theta(n \log n)$. But if the array is already sorted and the pivot is always the last element, every partition splits off just one element, giving

$$T(n) = T(n - 1) + \Theta(n) = \Theta(n^2).$$

This worst case is why practical implementations choose the pivot carefully — via random selection, or the "median-of-three" heuristic (comparing first, middle, and last elements) — which makes the $O(n^2)$ case statistically improbable rather than eliminating it.

Problem-solving application. Quicksort's average-case efficiency and in-place memory footprint make it the default choice in most language standard libraries for general sorting (though many, like Java's, switch to insertion sort for small subarrays and use dual-pivot variants for further speedup). When solving algorithm-design problems, recognize quicksort's partition step itself as a reusable subroutine — it underlies the classic $O(n)$ "median/k-th smallest element" selection algorithm (quickselect), since you only need to recurse into the side of the partition containing the target rank.